

```
// Merged files from branch: main
```

```
// --- Start of pubspec.yaml ---
```

```
name: cl837_accelerometer
description: CL837 Accelerometer data reader
publish_to: 'none'
version: 1.0.0+1
```

```
environment:
```

```
  sdk: '>=3.0.0 <4.0.0'
```

```
dependencies:
```

```
  flutter:
    sdk: flutter
  flutter_blue_plus: ^1.31.8
  permission_handler: ^11.0.1
  intl: ^0.18.0
  cupertino_icons: ^1.0.2
```

```
dev_dependencies:
```

```
  flutter_test:
    sdk: flutter
  flutter_lints: ^2.0.0
```

```
flutter:
```

```
  uses-material-design: true
```

```
// --- End of pubspec.yaml ---
```

```
// --- Start of lib\accelerometer_service.dart ---
```

```
import 'dart:async';
import 'package:flutter/foundation.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';
import 'battery.dart';
import 'heartrate.dart';
```

```
class AccelerometerData {
  final double x;
  final double y;
  final double z;
  const AccelerometerData({
    required this.x,
    required this.y,
    required this.z,
  });
}
```

```
class SensorService {
```

```
  // Accelerometer Service & Characteristics
```

```
  static const String _accelServiceUuid = 'aae28f00-71b5-42a1-8c3c-f9cf6ac969d0';
```

```
  static const String _accelDataCharUuid = 'aae28f01-71b5-42a1-8c3c-f9cf6ac969d0';
```

```
  static const double _scaleFactor = 8.0 / 32768.0; // For z8 g range
```

```
  final _accelDataStreamController = StreamController<AccelerometerData>.broadcast();
```

```
  final _heartRateStreamController = StreamController<int?>.broadcast();
```

```

final _batteryLevelStreamController = StreamController<int?>.broadcast();
final _stateStreamController = StreamController<bool>.broadcast();

StreamSubscription? _accelSubscription;
bool _isAccelerometerWorking = false;

Stream<AccelerometerData> get accelDataStream => _accelDataStreamController.stream;
Stream<int?> get heartRateStream => _heartRateStreamController.stream;
Stream<int?> get batteryLevelStream => _batteryLevelStreamController.stream;
Stream<bool> get connectionStream => _stateStreamController.stream;
bool get isAccelerometerWorking => _isAccelerometerWorking;

void _processAccelData(List<int> value, int? heartRate, int? batteryLevel) {
  if (value.length < 3) return;
  try {
    if (value[0] != 0xFF) return;
    if (value[2] == 0x0c) {
      for (var i = 3; i < value.length - 1; i += 6) {
        if (i + 5 >= value.length) break;
        final rawX = _convertToSigned16(value[i] | (value[i + 1] << 8));
        final rawY = _convertToSigned16(value[i + 2] | (value[i + 3] << 8));
        final rawZ = _convertToSigned16(value[i + 4] | (value[i + 5] << 8));
        final x = rawX * _scaleFactor;
        final y = rawY * _scaleFactor;
        final z = rawZ * _scaleFactor;
        _accelDataStreamController.add(AccelerometerData(x: x, y: y, z: z));
      }
    }
  } catch (e) {
    debugPrint('Error processing accelerometer data: $e');
  }
}

int _convertToSigned16(int value) {
  value &= 0xFFFF;
  return (value & 0x8000) != 0 ? -(0x10000 - value) : value;
}

Future<void> _startAccelerometer(BluetoothDevice device) async {
  try {
    debugPrint('\n=== STARTING ACCELEROMETER SETUP ===');
    final services = await device.discoverServices();
    // Setup Accelerometer
    final accelService = services.firstWhere(
      (s) => s.uuid.toString().toLowerCase() ==
_accelServiceUuid.toLowerCase(),
      orElse: () => throw Exception('Accelerometer service not found'),
    );
    debugPrint('Found Accelerometer service: ${accelService.uuid}');
    final accelChar = accelService.characteristics.firstWhere(
      (c) => c.uuid.toString().toLowerCase() ==
_accelDataCharUuid.toLowerCase(),
      orElse: () => throw Exception('Accelerometer characteristic not found'),
    );
  }
}

```

```

        debugPrint('Found Accelerometer characteristic: ${accelChar.uuid}');
        debugPrint('Accelerometer Properties: read=${accelChar.properties.read},
notify=${accelChar.properties.notify}');
        // Enable notifications
        final success = await accelChar.setNotifyValue(true);
        debugPrint('Accelerometer notifications enabled: $success');
        if (!success) {
            throw Exception('Failed to enable accelerometer notifications');
        }
        _accelSubscription = accelChar.lastValueStream.listen(
            (value) {
                debugPrint('Accelerometer data received: ${value.map((b) =>
'0x${b.toRadixString(16).padLeft(2, '0')}}').join(', ')}');
                _processAccelData(value, null, null); // Initial values without HR
and battery
            },
            onError: (error) {
                debugPrint('Accelerometer notification error: $error');
                _stateStreamController.add(false);
                _isAccelerometerWorking = false;
            },
        );
        _isAccelerometerWorking = true;
    } catch (e) {
        debugPrint('Accelerometer setup error: $e');
        _isAccelerometerWorking = false;
        rethrow;
    }
}

Future<void> _startServiceWithRetry(Future<void> Function() startFunction) async {
    const maxRetries = 3;
    for (var i = 0; i < maxRetries; i++) {
        try {
            await startFunction();
            return;
        } catch (e) {
            if (i == maxRetries - 1) {
                debugPrint('Service failed after $maxRetries retries: $e');
                rethrow;
            } else {
                debugPrint('Retrying service after error: $e');
                await Future.delayed(Duration(milliseconds: 500 * (i + 1)));
            }
        }
    }
}

```

```

Future<void> start(BluetoothDevice device, HeartRateService heartRateService,
BatteryService batteryService) async {
    try {
        // Start accelerometer service first - this is critical
        await _startAccelerometer(device);
        // Start heart rate and battery services in parallel with error handling
    }
}

```

```

    await Future.wait([
      _startServiceWithRetry(() => heartRateService.start(device))
        .catchError((e) {
          debugPrint('Heart rate service failed to start: $e');
          return null; // Continue without heart rate
        }),
      _startServiceWithRetry(() => batteryService.start(device))
        .catchError((e) {
          debugPrint('Battery service failed to start: $e');
          return null; // Continue without battery
        }),
    ], eagerError: false);
    // Set up subscriptions for heart rate and battery updates
    heartRateService.dataStream.listen((heartRate) {
      if (_accelSubscription != null && _isAccelerometerWorking) {
        // Update the sensor data with the new heart rate
        _heartRateStreamController.add(heartRate);
      }
    }, onError: (e) => debugPrint('Heart rate stream error: $e'));
    batteryService.dataStream.listen((battery) {
      if (_accelSubscription != null && _isAccelerometerWorking) {
        // Update the sensor data with the new battery level
        _batteryLevelStreamController.add(battery);
      }
    }, onError: (e) => debugPrint('Battery stream error: $e'));
    _stateStreamController.add(true);
  } catch (e) {
    debugPrint('Start error: $e');
    _stateStreamController.add(false);
    if (!_isAccelerometerWorking) {
      // Only rethrow if accelerometer failed
      rethrow;
    }
  }
}

Future<void> stop() async {
  debugPrint('Stopping BLE services...');
  await _accelSubscription?.cancel();
  _accelSubscription = null;
  _isAccelerometerWorking = false;
  _stateStreamController.add(false);
  debugPrint('All BLE services stopped');
}

void dispose() {
  debugPrint('Disposing BLE services...');
  stop();
  _accelDataStreamController.close();
  _heartRateStreamController.close();
  _batteryLevelStreamController.close();
  _stateStreamController.close();
  debugPrint('BLE services disposed');
}

```

```

}

// --- End of lib\accelerometer_service.dart ---

// --- Start of lib\battery.dart ---
import 'dart:async';
import 'package:flutter/foundation.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';

class BatteryServiceException implements Exception {
  final String message;
  BatteryServiceException(this.message);

  @override
  String toString() => 'BatteryServiceException: $message';
}

class BatteryService {
  // Battery Service & Characteristic UUIDs - shortened version
  static const String _batteryServiceUuid = '180f';
  static const String _batteryCharUuid = '2a19';

  int? _lastBatteryLevel;
  final _dataStreamController = StreamController<int?>.broadcast();
  StreamSubscription? _batterySubscription;

  Stream<int?> get dataStream => _dataStreamController.stream;
  int? get lastBatteryLevel => _lastBatteryLevel;

  void _processBattery(List<int> value) {
    try {
      debugPrint('Raw battery data: ${value.map((b) =>
'0x${b.toRadixString(16).padLeft(2, '0')}}').join(', ')}');
      if (value.isEmpty) return;

      _lastBatteryLevel = value[0];
      debugPrint('Processed Battery Level: $_lastBatteryLevel%');
      _dataStreamController.add(_lastBatteryLevel);
    } catch (e) {
      debugPrint('Error processing battery data: $e');
    }
  }

  Future<void> start(BluetoothDevice device) async {
    try {
      debugPrint('Setting up Battery service...');
      await Future.delayed(const Duration(milliseconds: 1000));

      final services = await device.discoverServices();
      debugPrint('Found ${services.length} services:');
      for (var service in services) {
        debugPrint('Service: ${service.uuid}');
        for (var char in service.characteristics) {
          debugPrint('Char: ${char.uuid}');

```

```

        debugPrint('Properties: Read=${char.properties.read},
Notify=${char.properties.notify}');
    }
}

final batteryService = services.firstWhere(
    (s) =>
s.uuid.toString().toLowerCase().contains(_batteryServiceUuid.toLowerCase()),
    orElse: () => throw BatteryServiceException('Battery service not found'),
);

debugPrint('Found Battery service: ${batteryService.uuid}');
final batteryChar = batteryService.characteristics.firstWhere(
    (c) => c.uuid.toString().toLowerCase().contains(_batteryCharUuid.toLowerCase()),
    orElse: () => throw BatteryServiceException('Battery characteristic not found'),
);

debugPrint('Found Battery characteristic: ${batteryChar.uuid}');
    debugPrint('Battery Properties: read=${batteryChar.properties.read},
notify=${batteryChar.properties.notify}');

// Try initial read if supported
if (batteryChar.properties.read) {
    try {
        final value = await batteryChar.read().timeout(
            const Duration(seconds: 2),
            onTimeout: () => throw TimeoutException('Battery read timeout')
        );
        debugPrint('Initial battery read successful');
        _processBattery(value);
    } catch (e) {
        debugPrint('Battery initial read failed: $e');
    }
}

// Enable notifications if supported
if (batteryChar.properties.notify) {
    try {
        final success = await batteryChar.setNotifyValue(true);
        debugPrint('Battery notifications enabled: $success');

        if (!success) {
            throw BatteryServiceException('Failed to enable battery notifications');
        }

        _batterySubscription = batteryChar.lastValueStream.listen(
            (value) {
                try {
                    debugPrint('Battery notification received');
                    _processBattery(value);
                } catch (e) {
                    debugPrint('Battery notification processing error: $e');
                }
            },

```

```

        onError: (e) {
          debugPrint('Battery notification stream error: $e');
        },
      );
    } catch (e) {
      debugPrint('Error setting up battery notifications: $e');
    }
  } else {
    debugPrint('Battery characteristic does not support notify');
  }

  debugPrint('Battery service setup complete');
} catch (e) {
  debugPrint('Battery service start failed: $e');
  _lastBatteryLevel = null;
  rethrow;
}
}

Future<void> stop() async {
  debugPrint('Stopping Battery service...');
  await _batterySubscription?.cancel();
  _batterySubscription = null;
  _lastBatteryLevel = null;
  debugPrint('Battery service stopped');
}

void dispose() {
  debugPrint('Disposing Battery service...');
  stop();
  _dataStreamController.close();
  debugPrint('Battery service disposed');
}
}

// --- End of lib\battery.dart ---

// --- Start of lib\heartrate.dart ---
import 'dart:async';
import 'package:flutter/foundation.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';

class HeartRateServiceException implements Exception {
  final String message;
  HeartRateServiceException(this.message);

  @override
  String toString() => 'HeartRateServiceException: $message';
}

class HeartRateService {
  // Heart Rate Service & Characteristic UUIDs - shortened version
  static const String _heartRateServiceUuid = '180d';
  static const String _heartRateCharUuid = '2a37';

```

```

int? _lastHeartRate;
final _dataStreamController = StreamController<int?>.broadcast();
StreamSubscription? _heartRateSubscription;

Stream<int?> get dataStream => _dataStreamController.stream;
int? get lastHeartRate => _lastHeartRate;

void _processHeartRate(List<int> value) {
  try {
    debugPrint('Raw heart rate data:  ${value.map((b) =>
'0x${b.toRadixString(16).padLeft(2, '0')}}').join(', ')}');
    if (value.isEmpty) return;

    // First byte contains flags
    final flags = value[0];
    final isFormat16Bit = (flags & 0x01) != 0; // Check first bit
    debugPrint('Heart Rate flags: 0x${flags.toRadixString(16)} (16-bit format:
$isFormat16Bit)');

    // Heart rate measurement value format
    if (value.length >= 2) {
      if (isFormat16Bit && value.length >= 3) {
        // 16-bit format
        _lastHeartRate = value[1] | (value[2] << 8);
      } else {
        // 8-bit format
        _lastHeartRate = value[1];
      }
      debugPrint('Processed Heart Rate: $_lastHeartRate BPM');
      _dataStreamController.add(_lastHeartRate);
    }
  } catch (e) {
    debugPrint('Error processing heart rate data: $e');
  }
}

Future<void> start(BluetoothDevice device) async {
  try {
    debugPrint('Setting up Heart Rate service...');
    await Future.delayed(const Duration(milliseconds: 1000));

    final services = await device.discoverServices();
    debugPrint('Found ${services.length} services:');
    for (var service in services) {
      debugPrint('Service: ${service.uuid}');
      for (var char in service.characteristics) {
        debugPrint('  Char: ${char.uuid}');
        debugPrint('    Properties:  Read=${char.properties.read},
Notify=${char.properties.notify}');
      }
    }

    final heartRateService = services.firstWhere(

```

(s)

=>


```

s.uuid.toString().toLowerCase().contains(_heartRateServiceUuid.toLowerCase()),
    orElse: () => throw HeartRateServiceException('Heart Rate service not found'),
);

debugPrint('Found Heart Rate service: ${heartRateService.uuid}');
final heartRateChar = heartRateService.characteristics.firstWhere(
                                                                    (c) =>
c.uuid.toString().toLowerCase().contains(_heartRateCharUuid.toLowerCase()),
    orElse: () => throw HeartRateServiceException('Heart Rate characteristic not
found'),
);

debugPrint('Found Heart Rate characteristic: ${heartRateChar.uuid}');
    debugPrint('Heart Rate Properties: read=${heartRateChar.properties.read},
notify=${heartRateChar.properties.notify}');

// Heart rate measurement only supports notifications
if (heartRateChar.properties.notify) {
    try {
        final success = await heartRateChar.setNotifyValue(true);
        debugPrint('Heart Rate notifications enabled: $success');

        if (!success) {
            throw HeartRateServiceException('Failed to enable heart rate
notifications');
        }

        _heartRateSubscription = heartRateChar.lastValueStream.listen(
            (value) {
                try {
                    debugPrint('Heart Rate notification received');
                    _processHeartRate(value);
                } catch (e) {
                    debugPrint('Heart Rate notification processing error: $e');
                }
            },
            onError: (e) {
                debugPrint('Heart Rate notification stream error: $e');
            },
        );

        debugPrint('Heart Rate subscription set up successfully');
    } catch (e) {
        debugPrint('Error setting up heart rate notifications: $e');
        rethrow;
    }
} else {
    throw HeartRateServiceException('Heart Rate characteristic does not support
notifications');
}

debugPrint('Heart Rate service setup complete');
} catch (e) {
    debugPrint('Heart Rate service start failed: $e');
}

```

```

        _lastHeartRate = null;
        rethrow;
    }
}

Future<void> stop() async {
    debugPrint('Stopping Heart Rate service...');
    await _heartRateSubscription?.cancel();
    _heartRateSubscription = null;
    _lastHeartRate = null;
    debugPrint('Heart Rate service stopped');
}

void dispose() {
    debugPrint('Disposing Heart Rate service...');
    stop();
    _dataStreamController.close();
    debugPrint('Heart Rate service disposed');
}
}
// --- End of lib\heartrate.dart ---

// --- Start of lib\main.dart ---
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';
import 'package:permission_handler/permission_handler.dart';
import 'accelerometer_service.dart';
import 'battery.dart';
import 'heartrate.dart';
import 'widgets/accelerometer_widget.dart';
import 'widgets/battery_widget.dart';
import 'widgets/heart_rate_widget.dart';

void main() {
    WidgetsFlutterBinding.ensureInitialized();
    FlutterBluePlus.setLogLevel(LogLevel.verbose, color: true);
    runApp(const MyApp());
}

class MyApp extends StatelessWidget {
    const MyApp({super.key});

    @override
    Widget build(BuildContext context) {
        return MaterialApp(
            debugShowCheckedModeBanner: false,
            title: 'CL837 Sensor Display',
            theme: ThemeData(
                primarySwatch: Colors.blue,
                visualDensity: VisualDensity.adaptivePlatformDensity,
            ),
            home: const SensorDisplayPage(),
        );
    }
}

```

```

    }
}

class SensorDisplayPage extends StatefulWidget {
    const SensorDisplayPage({super.key});

    @override
    State<SensorDisplayPage> createState() => _SensorDisplayPageState();
}

class _SensorDisplayPageState extends State<SensorDisplayPage> {
    final SensorService _sensorService = SensorService();
    final HeartRateService _heartRateService = HeartRateService();
    final BatteryService _batteryService = BatteryService();
    static const String targetDeviceName = 'CL837-0753644';
    BluetoothDevice? connectedDevice;
    AccelerometerData? latestAccelData;
    int? latestHeartRate;
    int? latestBatteryLevel;
    bool isScanning = false;
    bool isConnecting = false;
    late StreamSubscription<AccelerometerData> _accelDataSubscription;
    late StreamSubscription<int?> _heartRateSubscription;
    late StreamSubscription<int?> _batteryLevelSubscription;

    @override
    void initState() {
        super.initState();
        _initializeBluetooth();
    }

    Future<void> _initializeBluetooth() async {
        await _requestPermissions();
        await FlutterBluePlus.turnOn();
    }

    Future<void> _requestPermissions() async {
        await Future.wait([
            Permission.bluetooth.request(),
            Permission.bluetoothScan.request(),
            Permission.bluetoothConnect.request(),
            Permission.location.request(),
        ]);
    }

    Future<void> startScan() async {
        if (isScanning) return;
        setState(() {
            isScanning = true;
        });
        try {
            await FlutterBluePlus.startScan(timeout: const Duration(seconds: 5));
            FlutterBluePlus.scanResults.listen((results) {
                for (ScanResult r in results) {

```

```

        // ignore: deprecated_member_use
        debugPrint('Found device: ${r.device.name}');
        // ignore: deprecated_member_use
        if (r.device.name == targetDeviceName) {
            connectToDevice(r.device);
            FlutterBluePlus.stopScan();
            break;
        }
    }
});
await Future.delayed(const Duration(seconds: 5));
if (mounted) {
    setState(() {
        isScanning = false;
    });
}
} catch (e) {
    debugPrint('Error during scan: $e');
    if (mounted) {
        setState(() {
            isScanning = false;
        });
    }
    showError('Error during scan: $e');
}
}

```

```

Future<void> connectToDevice(BluetoothDevice device) async {
    if (isConnecting) return;
    setState(() {
        isConnecting = true;
    });
    try {
        await device.connect(timeout: const Duration(seconds: 10));
        try {
            await _sensorService.start(device, _heartRateService, _batteryService);
        } catch (e) {
            if (!_sensorService.isAccelerometerWorking) {
                showError('Critical error: Accelerometer not working');
                await disconnectDevice();
                return;
            } else {
                showWarning('Some sensors may not work properly. Accelerometer is
still functional.');
```

```

        showWarning('Some sensor data may be temporarily unavailable');
    },
);
_heartRateSubscription = _heartRateService.dataStream.listen(
    (heartRate) {
        setState(() {
            latestHeartRate = heartRate;
        });
    },
    onError: (error) {
        debugPrint('Heart rate stream error: $error');
        showWarning('Heart rate data may be temporarily unavailable');
    },
);
_batteryLevelSubscription = _batteryService.dataStream.listen(
    (batteryLevel) {
        setState(() {
            latestBatteryLevel = batteryLevel;
        });
    },
    onError: (error) {
        debugPrint('Battery stream error: $error');
        showWarning('Battery data may be temporarily unavailable');
    },
);
if (mounted) {
    setState(() {
        connectedDevice = device;
        isConnected = false;
    });
}
} catch (e) {
    debugPrint('Connection error: $e');
    if (mounted) {
        setState(() {
            isConnected = false;
        });
    }
    showError('Failed to connect to device');
}
}

```

```

Future<void> disconnectDevice() async {
    try {
        await _accelDataSubscription.cancel();
        await _heartRateSubscription.cancel();
        await _batteryLevelSubscription.cancel();
        await _sensorService.stop();
        await _heartRateService.stop();
        await _batteryService.stop();
        await connectedDevice?.disconnect();
    } catch (e) {
        debugPrint('Error during disconnect: $e');
    }
}

```

```

    if (mounted) {
      setState(() {
        connectedDevice = null;
        latestAccelData = null;
        latestHeartRate = null;
        latestBatteryLevel = null;
      });
    }
  }

  @override
  void dispose() {
    _accelDataSubscription.cancel();
    _heartRateSubscription.cancel();
    _batteryLevelSubscription.cancel();
    _sensorService.dispose();
    _heartRateService.dispose();
    _batteryService.dispose();
    disconnectDevice();
    super.dispose();
  }

  void showWarning(String message) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text(message),
        backgroundColor: Colors.orange,
        duration: const Duration(seconds: 3),
      ),
    );
  }

  void showError(String message) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text(message),
        backgroundColor: Colors.red,
        duration: const Duration(seconds: 3),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('CL837 Sensor Display'),
        actions: [
          if (connectedDevice != null)
            IconButton(
              icon: const Icon(Icons.bluetooth_connected),
              tooltip: 'Disconnect',
            ),
        ],
      ),
    );
  }

```

```

        onPressed: disconnectDevice,
      ),
    ],
  ),
  body: Column(
    children: [
      Container(
        color: Colors.grey[100],
        padding: const EdgeInsets.all(16.0),
        child: Row(
          children: [
            Expanded(
              child: ElevatedButton.icon(
                icon: const Icon(Icons.search),
                label: Text(
                  isScanning ? 'Scanning...' :
                  isConnecting ? 'Connecting...' :
                  'Scan for Device',
                ),
                onPressed: (isScanning || isConnecting) ? null :
startScan,
              ),
            ),
          ],
        ),
      ),
      Expanded(
        child: Padding(
          padding: const EdgeInsets.all(16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              HeartRateWidget(latestData: latestHeartRate),
              const SizedBox(height: 8),
              BatteryWidget(latestData: latestBatteryLevel),
              const SizedBox(height: 16),
              AccelerometerWidget(latestData: latestAccelData),
            ],
          ),
        ),
      ),
    ],
  ),
);
}
}

```

// --- End of lib\main.dart ---

// --- Start of lib\models\sensor_data.dart ---

```

class SensorData {
  final DateTime timestamp;
  final AccelerometerData? accelerometer;
  final int? heartRate;
}

```

```

final int? batteryLevel;

const SensorData({
  required this.timestamp,
  this.accelerometer,
  this.heartRate,
  this.batteryLevel,
});

SensorData copyWith({
  DateTime? timestamp,
  AccelerometerData? accelerometer,
  int? heartRate,
  int? batteryLevel,
}) {
  return SensorData(
    timestamp: timestamp ?? this.timestamp,
    accelerometer: accelerometer ?? this.accelerometer,
    heartRate: heartRate ?? this.heartRate,
    batteryLevel: batteryLevel ?? this.batteryLevel,
  );
}

class AccelerometerData {
  final double x;
  final double y;
  final double z;

  const AccelerometerData({
    required this.x,
    required this.y,
    required this.z,
  });
}

// --- End of lib\models\sensor_data.dart ---

// --- Start of lib\widgets\accelerometer_widget.dart ---
import 'package:flutter/material.dart';
import '../accelerometer_service.dart';

class AccelerometerWidget extends StatelessWidget {
  final AccelerometerData? latestData;
  const AccelerometerWidget({Key? key, required this.latestData}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        const Text(
          'Accelerometer',
          style: TextStyle(
            fontSize: 18,

```



```

        fontWeight: FontWeight.bold,
      ),
    ),
    const SizedBox(height: 8),
    if (latestData != null) ...[
      _buildAxisRow('X', latestData!.x, Colors.red),
      const SizedBox(height: 8),
      _buildAxisRow('Y', latestData!.y, Colors.green),
      const SizedBox(height: 8),
      _buildAxisRow('Z', latestData!.z, Colors.blue),
    ] else
      const Center(
        child: Text(
          'Waiting for data...',
          style: TextStyle(
            fontSize: 16,
            color: Colors.grey,
          ),
        ),
      ),
    ],
  );
}

```

```

Widget _buildAxisRow(String axis, double value, Color color) {
  return Card(
    elevation: 2,
    child: Padding(
      padding: const EdgeInsets.all(12.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Container(
                width: 24,
                height: 24,
                decoration: BoxDecoration(
                  // ignore: deprecated_member_use
                  color: color.withOpacity(0.2),
                  borderRadius: BorderRadius.circular(4),
                ),
                child: Center(
                  child: Text(
                    axis,
                    style: TextStyle(
                      color: color,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                ),
              ),
            ],
          ),
          const SizedBox(width: 12),
          Expanded(

```

```

        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            LinearProgressIndicator(
              value: (value + 8.0) / 16.0, //
              // ignore: deprecated_member_use
              backgroundColor: color.withOpacity(0.1),
              valueColor:
                AlwaysStoppedAnimation<Color>(color),
            ),
            const SizedBox(height: 4),
            Text(
              '${value.toStringAsFixed(3)} g',
              style: TextStyle(
                color: color,
                fontWeight: FontWeight.w500,
              ),
            ),
          ],
        ),
      ],
    ),
  ),
);
}
}

```

// --- End of lib\widgets\accelerometer_widget.dart ---

// --- Start of lib\widgets\battery_widget.dart ---

```
import 'package:flutter/material.dart';
```

```

class BatteryWidget extends StatelessWidget {
  final int? latestData;
  const BatteryWidget({Key? key, required this.latestData}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return _buildStatusRow(
      'Battery',
      latestData != null ? '$latestData%' : 'N/A',
      Icons.battery_full,
      color: _getBatteryColor(latestData),
    );
  }

  Widget _buildStatusRow(String label, dynamic value, IconData icon, {Color? color}) {
    return Card(
      elevation: 2,
      child: Padding(

```

```

padding: const EdgeInsets.all(12.0),
child: Row(
  children: [
    Icon(icon, size: 24, color: color),
    const SizedBox(width: 12),
    Text(
      label,
      style: const TextStyle(
        fontSize: 16,
        fontWeight: FontWeight.bold,
      ),
    ),
    const Spacer(),
    Text(
      value?.toString() ?? 'N/A',
      style: TextStyle(
        fontSize: 16,
        color: color,
        fontWeight: FontWeight.bold,
      ),
    ),
  ],
),
),
);
}

Color _getBatteryColor(int? level) {
  if (level == null) return Colors.grey;
  if (level > 60) return Colors.green;
  if (level > 30) return Colors.orange;
  return Colors.red;
}

// --- End of lib\widgets\battery_widget.dart ---

// --- Start of lib\widgets\heart_rate_widget.dart ---
import 'package:flutter/material.dart';

class HeartRateWidget extends StatelessWidget {
  final int? latestData;
  const HeartRateWidget({Key? key, required this.latestData}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return _buildStatusRow(
      'Heart Rate',
      latestData != null ? '$latestData BPM' : 'N/A',
      Icons.favorite,
      color: Colors.red,
    );
  }
}

```

```

Widget _buildStatusRow(String label, dynamic value, IconData icon, {Color? color}) {
  return Card(
    elevation: 2,
    child: Padding(
      padding: const EdgeInsets.all(12.0),
      child: Row(
        children: [
          Icon(icon, size: 24, color: color),
          const SizedBox(width: 12),
          Text(
            label,
            style: const TextStyle(
              fontSize: 16,
              fontWeight: FontWeight.bold,
            ),
          ),
          const Spacer(),
          Text(
            value?.toString() ?? 'N/A',
            style: TextStyle(
              fontSize: 16,
              color: color,
              fontWeight: FontWeight.bold,
            ),
          ),
        ],
      ),
    ),
  );
}

// --- End of lib\widgets\heart_rate_widget.dart ---

```